

Documentație bloc de calcul

0. Reprezentarea în virgulă fixă, Q16.16

Pentru proiectul acesta, avem nevoie să reprezentăm într-un anumit mod coordonatele punctelor, de tip numere reale. În acest scop, putem folosi:

- fixed-point, folosește un număr fix de biți pentru partea întreagă, respectiv partea fracțională a numărului
- floating-point, folosește notația științifică (cu exponent/mantisa) ce deplasează dinamic virgula, permițând o gamă mai largă de valori, în detrimentul performanței

Deoarece precizia scade cu cât numerele devin mai mari în format floating-point, iar această reprezentare necesită mai multe resurse decât ar fi necesar pentru FPGA-ul folosit, am decis să folosesc reprezentarea fixed-point.

Pentru un număr I de biți pentru partea întreagă (cu semn) și F biți pentru cea fracțională, un număr fixed-point semnat este reprezentat pe $I + F$ biți, unde:

- bitul $[I+F-1]$ - bitul de semn
- biții $[I+F-2:F]$ - partea întreagă
- biții $[F-1:0]$ - partea fracțională

Pentru codul proiectului am ales să folosesc formatul Q16.16, unde:

- bitul $[31]$ - bitul de semn
- biții $[30:16]$ - partea întreagă a numărului
- biții $[15:0]$ - partea fracționară a numărului

Observație: În notația Q, bitul de semn este inclus în **INT_BITS**. Astfel, pentru Q16.16, din cei 16 biți alocați părții întregi, bitul $[31]$ este bitul de semn, iar biții $[30:16]$ reprezintă efectiv partea întreagă — rezultând 15 biți utili pentru magnitudine. De aceea intervalul reprezentabil al părții întregi este asimetric: $[-32768, +32767]$, iar valoarea maximă **totală** în format Q16.16 este $[-32768, +32767.99998474121]$

La modul general, valorile maxime și minime în format Q sunt parametrizabile astfel:

- $\text{localparam } [TOTAL-1:0] \text{ MAX} = \{1'b0, \{(TOTAL-1)\{1'b1\}\}\};$
- $\text{localparam } [TOTAL-1:0] \text{ MIN} = \{1'b1, \{(TOTAL-1)\{1'b0\}\}\};$

unde $TOTAL = INT_BITS + FRAC_BITS$ ($I+F$)

În format Q16.16, valoarea maximă reprezentabilă este **MAX = 7FFF_FFFF** (+32767.99998474121 în zecimal), iar valoarea minimă este **MIN = 8000_0000** (-32768 în zecimal)
Rezoluția este **RES = 0000_0001**, sau $2^{-16} = 0.0000152587890625$

Vom vedea ulterior în simulări ce se întâmplă când modificăm formatul Q, de aceea următoarele au numere de biți parametrizabile.

1. Componente aritmetice

1.1. Modul adunare *add_q.v*

Modulul **add_q** implementează adunarea a două numere fixed-point în format Q parametrizabil, cu detecție de overflow și saturare.

Modulul este sincron pe semnalul de ceas și produce rezultatul într-un registru pentru a permite integrarea modulului într-un pipeline aritmetic fără a introduce bucle combinatoriale lungi.

Semnal	Direcție	Lățime biți	Descriere
clk	input	1	semnal de ceas
rst_n	input	1	reset asincron, activ în 0
a	input	I+F	operand Q
b	input	I+F	operand Q
sum	output	I+F	rezultat Q saturat
overflow	output	1	flag overflow

Suma propriu-zisă este reprezentată de firul **raw_sum**: **raw_sum = a + b**, urmând apoi a fi verificată condiția de overflow.

Cum detectăm overflow la adunare ?

Pentru numere de tip signed (reprezentate în complement față de 2), dacă operanzii au *același semn*, iar semnul rezultatului diferă de semnul oricărui operand, atunci știm că suma s-a extins spre bitul de semn, cauzând starea de overflow.

Exemplu: Fie numerele de tip signed pe 5 biți 01011 (+11) și 00111 (+7)

Suma va fi 10010, însă deoarece noi folosim reprezentarea în complement față de 2, la final, programul va interpreta rezultatul ca fiind -14. Rezultatul actual (18) are nevoie de un bit suplimentar de date...

Logica de detecție overflow este:

$\text{add_overflow} = (\sim(a[\text{TOTAL}-1] \wedge b[\text{TOTAL}-1])) \& (\text{raw_sum}[\text{TOTAL}-1] \wedge a[\text{TOTAL}-1])$

Condiția 1: $\sim(a[\text{MSB}] \wedge b[\text{MSB}])$ – a și b au același semn

Dacă au semne diferite, rezultatul e mereu în interval — nu poate da overflow:

$(+) + (-) \rightarrow$ rezultat mai mic decât cel mai mare operand

$(-) + (+) \rightarrow$ rezultat mai mare decât cel mai mic operand

Dacă au același semn, poți ieși din interval:

$(+) + (+) \rightarrow$ poate depăși MAX

$(-) + (-) \rightarrow$ poate coborî sub MIN

Condiția 2: $\text{raw_sum}[\text{MSB}] \wedge a[\text{MSB}]$ (sau b, rezultatul va fi la fel) – semnul rezultatului diferă de semnul oricărui operand

Dacă ambii sunt pozitivi, rezultatul ar trebui să fie pozitiv. Dacă a ieșit negativ, s-a produs wrap-around, deci overflow.

La final, dacă cele 2 condiții sunt satisfăcute, **add_overflow** va returna 1.

Pentru a rezolva această problemă, avem la dispoziție 2 soluții:

1. **Wrap-around:** rezultatul pur și simplu continuă mai departe, începând cu cea mai mică valoare posibilă, deoarece datele se extind spre bitul de semn. Deși nu necesită logică extra, rezultatul nu mai reprezintă suma matematică reală din cauza wrap-around-ului în complement față de 2.

$0111 (+7) + 0001 (+1) = 1000 (-8)$ (sare direct la minim)

$1111 (-1) + 0001 (+1) = 0000$ (resetare biți de date)

2. **Saturație** (soluția pe care am și adoptat-o): Rezultatul este setat la MAX sau MIN după depășire. Avantajul cel mai mare îl reprezintă rezultatul cel mai apropiat de adevăr, chiar dacă necesită logica de detecție și fixare !

$0111 (+7) + 0001 (+1) = 0111 (+7)$ (rămâne la MAX)

$1000 (-8) + 1111 (-1) = 1000 (-8)$ (rămâne la MIN)

Saturația, și implicit rezultatul final, este satisfăcută de linia:

sum <= add_overflow ? (a[TOTAL-1] ? MIN : MAX) : raw_sum;

...sau:

DACĂ (add_overflow = 1)

DACĂ (a[TOTAL-1] = 1)

sum = MIN

ALTFEL

sum = MAX

ALTFEL

sum = raw_sum

Practic, setăm suma finală la MAX/MIN în funcție de flag-ul overflow și bitul de semn al oricărui operator, sau la suma brută în caz contrar.

La final, rezultatul este actualizat pe frontul crescător al semnalului **clk**. După reset, ieșirile **sum** și **overflow** sunt setate la 0.

Câteva exemple:

a	b	Interpretare	sum	overflow
0001_0000	0001_0000	$1.0 + 1.0 = 2.0$	0002_0000	0
0000_8000	0000_8000	$0.5 + 0.5 = 1.0$	0001_0000	0
7FFF_FFFF	0000_0001	$\text{MAX} + \text{RES} = \text{MAX}$	7FFF_FFFF	1
8000_0000	FFFF_FFFF	$\text{MIN} + (-\text{RES}) = \text{MIN}$	8000_0000	1
FFFF_0000	0001_0000	$-1.0 + 1.0 = 0.0$	0000_0000	0
0000_0001	FFFF_FFFF	$2^{16} + (-\text{RES}) = 0.0$	0000_0000	0
7FFF_0000	0001_0000	$32767.0 + 1.0 = \text{MAX}$	7FFF_FFFF	1
4000_0000	4000_0000	$16384.0 + 16384.0 = \text{MAX}$	7FFF_FFFF	1

8000_0000	FFFF_FFFF	MIN + (-RES) = MIN	8000_0000	1
-----------	-----------	--------------------	-----------	---

1.2. Modul scădere *sub_q.v*

Modulul **sub_q** implementează scăderea a două numere fixed-point în format Q parametrizabil, cu detecție de overflow și saturare.

Modulul este sincron pe semnalul de ceas și produce rezultatul într-un registru pentru a permite integrarea modulului într-un pipeline aritmetic fără a introduce bucle combinatoriale lungi.

Semnal	Direcție	Lățime biți	Descriere
clk	input	1	semnal de ceas
rst_n	input	1	reset asincron, activ în 0
a	input	I+F	operand Q
b	input	I+F	operand Q
dif	output	I+F	rezultat Q saturat
overflow	output	1	flag overflow

Diferența brută este reprezentată de firul **raw_dif**: $\text{raw_dif} = a - b$, urmând apoi a fi verificată condiția de overflow.

Cum detectăm overflow la scădere ?

Aceeași logică ca la detecția overflow la adunare, dar de data aceasta trebuie să verificăm că respectivii operatori au semne *diferite*.

Logica de overflow la scădere este:

$\text{sub_overflow} = (a[\text{TOTAL}-1] \wedge b[\text{TOTAL}-1]) \& (\text{raw_dif}[\text{TOTAL}-1] \wedge a[\text{TOTAL}-1])$

Condiția 1: $a[\text{MSB}] \wedge b[\text{MSB}]$ – a și b au semne diferite

Dacă au același semn, rezultatul e mereu în interval — nu poate da overflow:

(+) - (+) : rezultat mic

(-) - (-) : rezultat mic

Dacă au semne diferite, scăzând un negativ din pozitiv (sau invers) poți ieși din interval:

$(+) - (-) = (+) + (+)$: poate depăși MAX

$(-) - (+) = (-) + (-)$: poate coborî sub MIN

Condiția 2: $\text{raw_dif}[\text{MSB}] \wedge \text{a}[\text{MSB}]$ – semnul rezultatului diferă de semnul lui **a**

*Dacă **a** e pozitiv, rezultatul ar trebui să fie pozitiv (sau cel mult să fi depășit MAX). Dacă a ieșit negativ, s-a produs wrap-around, deci overflow.*

Saturația, și implicit rezultatul final, este satisfăcută de linia:

$\text{dif} \leq \text{sub_overflow} ? (\text{a}[\text{TOTAL}-1] ? \text{MIN} : \text{MAX}) : \text{raw_dif};$

...sau:

DACĂ ($\text{sub_overflow} = 1$)

DACĂ ($\text{a}[\text{TOTAL}-1] = 1$)

$\text{dif} = \text{MIN}$

ALTFEL

$\text{dif} = \text{MAX}$

ALTFEL

$\text{dif} = \text{raw_dif}$

La final, rezultatul este actualizat pe frontul crescător al semnalului **clk**. După reset, ieșirile **dif** și **overflow** sunt setate la 0.

Exemple:

a	b	Interpretare	dif	overflow
0002_0000	0001_0000	$2.0 - 1.0 = 1.0$	0001_0000	0
0001_0000	0001_0000	$1.0 - 1.0 = 0.0$	0000_0000	0
0000_0001	0000_0001	$\text{RES} - \text{RES} = 0.0$	0000_0000	0
7FFF_FFFF	FFFF_0000	$\text{MAX} - (-1.0) = \text{MAX}$	7FFF_FFFF	1
7FFF_0000	FFFF_0000	$32767.0 - (-1.0) = \text{MAX}$	7FFF_FFFF	1
8000_0000	0001_0000	$\text{MIN} - 1.0 = \text{MIN}$	8000_0000	1
8000_0000	0000_0001	$\text{MIN} - \text{RES} = \text{MIN}$	8000_0000	1
FFFF_0000	0001_0000	$-1.0 - 1.0 = -2.0$	FFFE_0000	0

4000_0000	C000_0000	16384.0 – (-16384.0) = MAX	7FFF_FFFF	1
-----------	-----------	----------------------------	-----------	---

1.3. Modul înmulțire *mult_q.v*

Modulul **mult_q** implementează înmulțirea a două numere fixed-point în format Q parametrizabil, cu detecție de overflow și saturare.

Modulul este sincron pe semnalul de ceas și produce rezultatul într-un registru pentru a permite integrarea modulului într-un pipeline aritmetic fără a introduce bucle combinatoriale lungi.

Semnal	Direcție	Lățime biți	Descriere
clk	input	1	semnal de ceas
rst_n	input	1	reset asincron, activ în 0
a	input	I+F	operand Q
b	input	I+F	operand Q
result	output	I+F	rezultat Q saturat
overflow	output	1	flag overflow

Pentru a evita utilizarea tipului *signed*, operandii sunt transformați mai întâi în **valori absolute**, asigurate de firele **abs_a** și **abs_b**, iar semnul rezultatului este determinat separat.

Semnul produsului este determinat cu ajutorul unei porți XOR aplicate pe biții de semn ai operandilor:

$$\text{result_sign} = a[\text{TOTAL}-1] \wedge b[\text{TOTAL}-1]$$

Dacă operandii au semne diferite, rezultatul va fi negativ. Operatorii sunt convertiți în valori absolute folosind reprezentarea în **complement față de 2**:

$$\text{abs_a} = a[\text{TOTAL}-1] ? (\sim a + 1) : a$$

$$\text{abs_b} = b[\text{TOTAL}-1] ? (\sim b + 1) : b$$

Dacă bitul de semn este 1, numărul este negativ și se calculează valoarea absolută.

Produsul brut este asigurat de firul **raw_product = abs_a * abs_b**. Trebuie ținut cont că produsul dintre 2 numere pe N biți se va extinde pe 2*N biți:

a: INT_BITS . FRAC_BITS (virgula la poziția FRAC_BITS)
b: INT_BITS . FRAC_BITS (virgula la poziția FRAC_BITS)
*raw_product: 2*INT_BITS . 2*FRAC_BITS (virgula la poziția 2*FRAC_BITS)*

Pentru a reveni la formatul Q dorit, rezultatul trebuie deplasat la dreapta cu **FRAC_BITS**:

shifted = raw_product >> FRAC_BITS

După această operație, rezultatul intermediar se află pe **2*TOTAL** biți, cu

*shifted: 2*INT_BITS . FRAC_BITS (virgula la poziția FRAC_BITS)*

dar doar **TOTAL** biți pot fi reprezentați în formatul final.

Cum detectăm overflow la înmulțire ?

Overflow apare atunci când rezultatul nu poate fi reprezentat în intervalul MIN-MAX; cu toate acestea, verificarea depinde de semnul rezultatului.

OVERFLOW POZITIV

Într-un număr pozitiv pe **TOTAL** biți, bitul de semn trebuie să fie 0 și nu trebuie să existe biți de 1 mai sus. Verificarea se implementează prin:

pos_overflow = !result_sign && (|shifted[2*TOTAL-1 : TOTAL-1])

Operatorul | calculează bit-cu-bit funcția OR între toți biții **[2*TOTAL-1 : TOTAL-1]**. Dacă oricare din acești biți este 1, magnitudinea nu încapă în **TOTAL** biți semnați. Includem bitul **TOTAL-1** în test deoarece, pentru un rezultat pozitiv, acesta trebuie să fie 0 — altfel ar fi interpretat ca sign bit negativ.

OVERFLOW NEGATIV

Dacă rezultatul ar trebui să fie negativ (**result_sign = 1**), verificăm dacă valoarea absolută depășește intervalul reprezentabil pentru numere negative. Pentru numere negative în complement față de 2, limita absolută este **MIN = 2^(TOTAL-1) (0x8000....0000)**. De aceea, verificarea este:

neg_overflow = result_sign && (|shifted[2*TOTAL-1 : TOTAL] || (shifted[TOTAL-1:0] > MIN))

- **|shifted[2*TOTAL-1 : TOTAL]** — există biți nenuli strict deasupra bitului **TOTAL-1**; magnitudinea depășește **TOTAL** biți, overflow clar

- $\text{shifted}[\text{TOTAL}-1:0] > \text{MIN}$ — toți biții de sus sunt 0, dar magnitudinea în **TOTAL** biți depășește $2^{(\text{TOTAL}-1)}$; pentru negativ putem reprezenta exact MIN (care corespunde $-2^{(\text{TOTAL}-1)}$), dar orice magnitudine mai mare nu încapă, deci overflow.

Diferența față de cazul pozitiv: pragul este **TOTAL-1** biți utili pentru pozitiv, respectiv **TOTAL** biți (cu magnitudinea $\leq \text{MIN}$) pentru negativ — asimetria clasică a complementului față de 2.

Overflow-ul final este obținut prin:

$\text{comb_overflow} = \text{pos_overflow} \parallel \text{neg_overflow}$

APLICAREA SEMNULUI

După calcularea produsului pe valori absolute, semnul este reaplicat. Dacă rezultatul trebuie să fie negativ, se aplică din nou complementul față de 2:

$\text{neg_result} = \sim\text{shifted}[\text{TOTAL}-1:0] + 1$

Rezultatul final devine:

$\text{final_result} = \text{result_sign} ? \text{neg_result} : \text{shifted}[\text{TOTAL}-1:0]$

APLICAREA SATURAȚIEI

Dacă apare overflow, rezultatul este saturat la valorile limită ale formatului Q16.16:

$\text{result} \leq \text{comb_overflow} ? (\text{result_sign} ? \text{MIN} : \text{MAX}) : \text{final_result}$

...sau:

DACĂ ($\text{comb_overflow} = 1$)

 DACĂ ($\text{result_sign} = 1$)

$\text{result} = \text{MIN}$

 ALTFEL

$\text{result} = \text{MAX}$

ALTFEL

$\text{result} = \text{final_result}$

La final, rezultatul este actualizat pe frontul crescător al semnalului **clk**. După reset, ieșirile **result** și **overflow** sunt setate la 0.

Exemple:

a	b	Interpretare	result	overflow
0002_0000	0001_0000	$2.0 * 1.0 = 2.0$	0002_0000	0
0001_0000	FFFF_0000	$1.0 * (-1.0) = -1.0$	FFFF_0000	0
7FFF_FFFF	0002_0000	$MAX * 2.0 = MAX$	7FFF_FFFF	1
8000_0000	0002_0000	$MIN * 2.0 = MIN$	8000_0000	1
0000_8000	0000_8000	$0.5 * 0.5$	0000_4000	0

1.4. Modul împărțire

1.5. Limitări

- modulele operează doar pe format Q16.16
- nu există rotunjire, doar trunchiere implicită
- operațiile durează 1 ciclu de ceas (mai puțin cea de împărțire)

2. Module formule matematice